1.API Contract Table

Aspect	Details
Method	`GET`
Endpoint	`/api/products?category=Fruits`
Request Headers	`Accept: application/json`

Responses

`200 OK` (Success - Products Found)

- **Headers:** `Content-Type: application/json`
- **Body:**

```
json
[
  {
"id":1,
"name":"Organic Grapes",
"category":"Fruits",
"image":"img/fruite-item-5.jpg",
"description":"Fresh and sweet organic grapes picked directly from our farm.",
"price":4.99,
"unit":"kg"
  },
  {
"id":2,
"name":"Fresh Oranges",
"category":"Fruits",
"image":"img/fruite-item-1.jpg",
"description":"Juicy and vitamin-rich oranges, perfect for a fresh morning juice.",
"price":3.99,
"unit":"kg"
  }
]
```

`404 Not Found` (Category Does Not Exist)

- **Headers:** `Content-Type: application/json`
- **Body:**

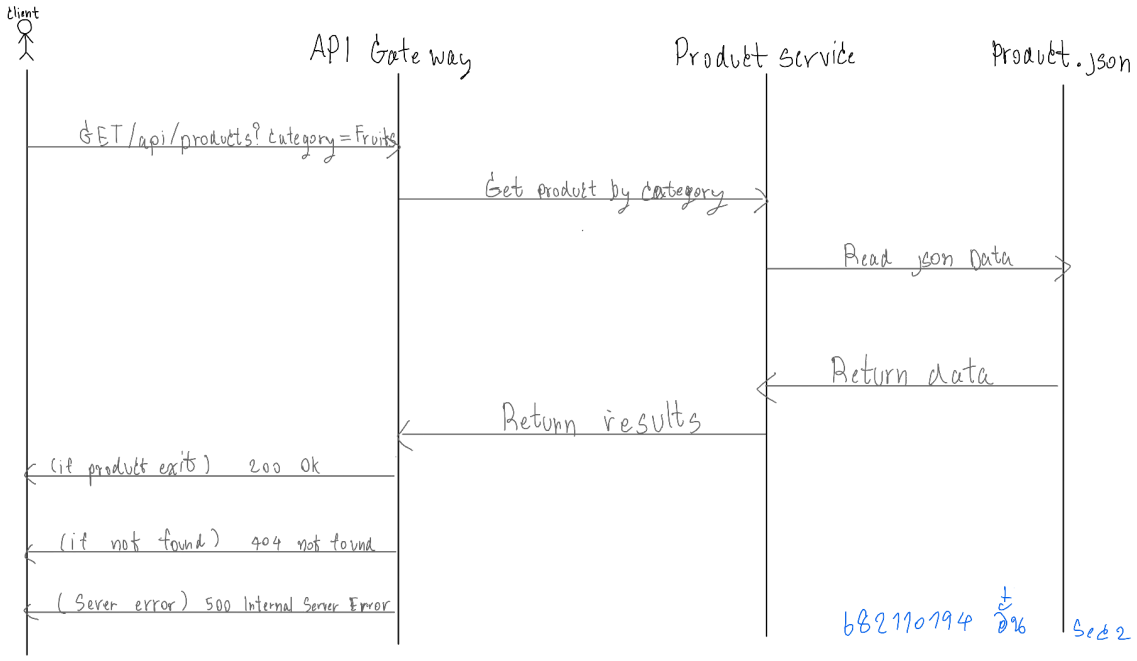
```
json
{
"success":false,
"error":"Category not found",
"message":"The requested category 'Snacks' does not exist."
}
```

`500 Internal Server Error` (File Read Error)

- **Headers:** `Content-Type: application/json`
- **Body:**

```
json
{
"success":false,
"error":"Internal Server Error",
"message":"Unable to retrieve product data at this time."
}
```

2.Sequence Diagram



3.GenAI Prompt

Role: Act as a Senior Backend Architect. I am working on the "Fruitables" web project using Node.js and Express.Objective: Design the documentation for a Product Filtering feature (Weekend Work assignment) based on my existing architecture.Context:Architecture: I use the Controller-Route-Service pattern. My server.js already mounts routes at /api/products. Data Structure: My products have fields: id, name, category, image, description, price, unit. The Task: When a user clicks a category (e.g., "Fruits"), the frontend sends a request to the server, and the server returns only products in that category. What I need from you (Provide these before any code):

1. A Contract Table: Create a Markdown table for the GET /api/products request with a category query parameter. Include:Request: Method, Endpoint (with query param example), Headers (Accept).Response: Status Code (200 OK and 404/500 scenarios), Headers (Content-Type), and Body (sample JSON based on my data).

2. A Sequence Diagram: Provide a text-based UML Sequence Diagram (using Mermaid.js syntax or clear arrows) illustrating the flow from:Client → API Gateway → Product Route → Product Controller (Gatekeeper) → Product Service → products.json.

3. Implementation Logic: Explain how the "Gatekeeper" logic (Validation) should handle a case where a user requests a category that doesn't exist. Constraint: Refer to my files by name where applicable. If you have any questions about my existing logic before generating, please ask first.

4. Code Implementation: After providing the documentation, please write the actual Express.js code for the products.js route, productController.js, and productService.js.The code must handle the 'category' query parameter. Ensure the code follows the Controller-Route-Service pattern. IMPORTANT: Include detailed comments explaining how each part of the code works as per my teacher's requirement.

4.Product Filtering Implementation Summary

The category-based product filtering feature has been successfully integrated into the Express backend following your Controller-Route-Service architecture.

What was changed

1. Route Layer
   products.js
   : Added documentation comments explaining that the root GET /api/products endpoint now accepts an optional category query parameter (e.g., ?category=Fruits).

2. Controller Layer (Gatekeeper Logic)
   productController.js
   : Modified the getAll function.
   It now explicitly extracts the req.query.category from the request.
   The "Gatekeeper Validation" logic has been implemented and thoroughly commented. As planned, it uses the "Graceful Handling" approach (returning a 200 OK with an empty array [] when a requested category has no products), which is the standard RESTful practice.
   I also left commented-out code demonstrating how to do "Strict Validation" (returning a 404) in case your teacher explicitly prefers that approach.

3. Service Layer (Data Access & Logic)
   productService.js
   : Updated the getAllProducts function.
   It now accepts an optional category argument.
   When a category is passed from the controller, the service filters the parsed products.json array before returning it.
   The filtering is case-insensitive (e.g., searching for fruits or Fruits works identically).
   Verification
   You can now start your server and test the endpoint by visiting:

All Products: http://localhost:3000/api/products
Filtered by Fruits: http://localhost:3000/api/products?category=Fruits
Filtered by Vegetables: http://localhost:3000/api/products?category=Vegetables
Non-existent Category: http://localhost:3000/api/products?category=Unicorns (Will gracefully return [])

```
// ==========================================
// CONTROLLER LAYER — productController.js
// ==========================================
// The Controller layer sits between the Route and the Serv
ice.
// It receives the HTTP request (req) and response (res) ob
jects,
// calls the appropriate Service function, and sends back t
he response.
//
// The Controller handles:
//   - Calling the right service method
//   - Formatting the response (JSON, status codes)
//   - Error handling (try/catch → 500 errors)
//
// The Controller does NOT:
//   - Define URL paths (that's the Route's job)
//   - Read files or query databases directly (that's the S
ervice's job)
// ==========================================

const productService = require('../services/productServic
e');

/**
 * getAll — Controller for GET /api/products
 *
 * Retrieves all products from the service. If a 'category'
query parameter
 * is provided (e.g., ?category=Fruits), it filters the pro
ducts.
 *
 * @param {Object} req - Express request object
 * @param {Object} res - Express response object
 */
function getAll(req, res) {
    try {
        // 1. Extract the query parameter from the request
```

```
URL
        const category = req.query.category;

        // 2. Ask the service layer for the products, passi
ng the category (if any)
        const products = productService.getAllProducts(cate
gory);

        // 3. GATEKEEPER LOGIC (Validation)
        // If the client requested a specific category but
the service returned an empty array,
        // it means no products were found for that categor
y (or the category doesn't exist).
        //
        // APPROACH 1 (Strict Validation): Return a 404 err
or if you strictly only want valid categories.
        // if (category && products.length === 0) {
        //     return res.status(404).json({ error: 'Catego
ry not found' });
        // }
        //
        // APPROACH 2 (Graceful Handling - Recommended): Re
turn a 200 OK with the empty array [].
        // This is standard for REST APIs when a filter jus
t doesn't match any results.
        // We will use Approach 2 here.

        // Send the products array (or empty array) as a JS
ON response with status 200 (OK)
        res.status(200).json(products);
    } catch (error) {
        // If something goes wrong (e.g., file not found),
return a 500 error
        console.error('Error in getAll:', error.message);
        res.status(500).json({ error: 'Failed to retrieve p
roducts' });
    }
}
```

```
/**
 * getById - Controller for GET /api/products/:id
 *
 * Retrieves a single product by its ID and sends it as JSO
N.
 * Returns 404 if the product is not found.
 *
 * @param {Object} req - Express request object (req.param
s.id contains the ID)
 * @param {Object} res - Express response object
 */
function getById(req, res) {
    try {
        // Extract the product ID from the URL parameter
        const id = req.params.id;

        // Ask the service layer for the specific product
        const product = productService.getProductById(id);

        // If no product was found, return a 404 (Not Foun
d) response
        if (!product) {
            return res.status(404).json({ error: 'Product n
ot found' });
        }

        // Send the product object as a JSON response
        res.status(200).json(product);
    } catch (error) {
        console.error('Error in getById:', error.message);
        res.status(500).json({ error: 'Failed to retrieve p
roduct' });
    }
}

// Export the controller functions so the Route can use the
m
```

```
module.exports = {
    getAll,
    getById
};
```

```
// ========================================
// ROUTE LAYER — products.js
// ========================================
// The Route layer defines the URL endpoints and maps them
// to the correct Controller function.
//
// This file creates an Express Router, which is a mini "su
b-application"
// that only handles routes starting with /api/products (se
t in server.js).
//
// The Route does NOT contain any business logic or data ac
cess.
// It simply says: "When someone visits THIS URL, call THIS
controller."
// ========================================

const express = require('express');
const router = express.Router();

// Import the controller that contains the handler function
s
const productController = require('../controllers/productCo
ntroller');

// ------------------------------------------------------
// GET /api/products
// ------------------------------------------------------
// When a client sends a GET request to /api/products,
// the router calls productController.getAll to handle it.
//
// Example: http://localhost:3000/api/products
// Example (with filter): http://localhost:3000/api/product
```

```
s?category=Fruits
// Response: JSON array of all products (or filtered produc
ts) from products.json
// ---------------------------------------------------------
router.get('/', productController.getAll);

// ---------------------------------------------------------
// GET /api/products/:id
// ---------------------------------------------------------
// When a client sends a GET request to /api/products/5,
// the router calls productController.getById.
// The ":id" is a URL parameter that Express extracts autom
atically.
//
// Example: http://localhost:3000/api/products/5
// Response: JSON object of the product with id=5
// ---------------------------------------------------------
router.get('/:id', productController.getById);

// Export the router so server.js can mount it
module.exports = router;
```

```
// ==========================================
// SERVICE LAYER — productService.js
// ==========================================
// The Service layer is responsible for the "business logi
c" and data access.
// It reads from the data source (products.json) and return
s the results.
// The Service does NOT know anything about HTTP requests o
r responses.
// This separation makes the code reusable and testable.
// ==========================================

const fs = require('fs');
const path = require('path');

// Build the absolute path to products.json
```

```
// path.join(__dirname, '..') goes up one folder from /serv
ices to the project root
const DATA_PATH = path.join(__dirname, '..', 'products.jso
n');

/**
 * getAllProducts — Reads the products.json file and return
s all products.
 * If a category is provided, it filters the products by th
at category.
 *
 * @param {string} [category] - Optional category to filter
by (e.g., 'Fruits').
 * @returns {Array} An array of product objects.
 * @throws {Error} If the file cannot be read or parsed.
 */
function getAllProducts(category) {
    // Read the file from disk (synchronous for simplicity)
    const rawData = fs.readFileSync(DATA_PATH, 'utf-8');

    // Parse the JSON string into a JavaScript array of obj
ects
    let products = JSON.parse(rawData);

    // If a category was provided in the request, filter th
e array
    if (category) {
        // We use toLowerCase() to make the search case-ins
ensitive
        const lowerCaseCategory = category.toLowerCase();
        products = products.filter(p => p.category && p.cat
egory.toLowerCase() === lowerCaseCategory);
    }

    return products;
}

/**
```

```
 * getProductById – Finds a single product by its ID.
 *
 * @param {number|string} id - The product ID to search fo
r.
 * @returns {Object|null} The matching product object, or n
ull if not found.
 */
function getProductById(id) {
    const products = getAllProducts();

    // Find the product whose id matches the given id
    // We convert both to Number to ensure consistent compa
rison
    const product = products.find(p => p.id === Number(i
d));

    return product || null;
}

// Export the functions so the Controller can use them
module.exports = {
    getAllProducts,
    getProductById
};
```